

Autonomous Competitive Intelligence Agent

Track: Loyalty Program Competitive Intelligence · Phase 2

TEAM MEMBERS

Shreyas R Gowda — Pipeline orchestration, extraction schema, reconciliation logic

Shreeram G Patgar — Retrieval layer, source handling, deduplication

Naveen G — Output layer (brief generation), evaluation, sentiment sourcing

Problem & Our Approach

💡 THE CORE CHALLENGE

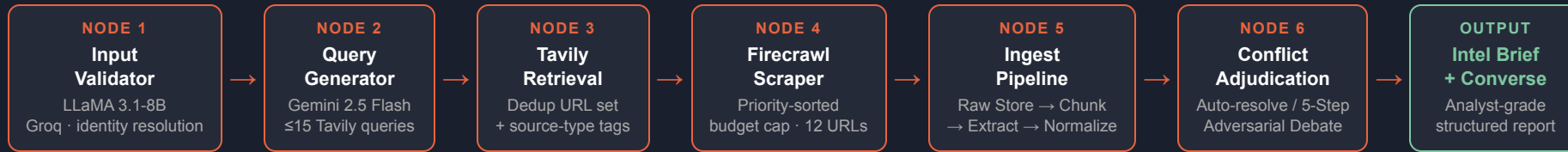
- Loyalty programs are documented **inconsistently** across official pages, T&C PDFs, review forums, and SEO blogs
- Manually researching one program to a competitive standard takes **4–8 hours** and the result goes stale quickly
- Sources contradict each other — earn rates, point values, and tier rules differ site-to-site; no existing tool surfaces these conflicts
- Programs rebrand and revise rules; without temporal awareness, intelligence decays silently

⚙️ OUR KEY INSIGHT

- Treat every extracted fact as a *claim with provenance* — not a value to overwrite
- Use an **adversarial debate engine** (Advocate A vs B → LLaMA Judge) to resolve conflicts transparently; the LLM can never invent a third value
- Separate retrieval, scraping, and extraction so each stage is independently replaceable
- Hard rule: `null` when unknown — never hallucinated
- Volatility weighting: earn rates weight recency; tier names weight authority

DATA QUALITY: Web coverage for smaller programs (regional airlines, fuel retailers) is sparse; sentiment fields from Reddit/Trustpilot can be noisy. We addressed this by tiering query budgets — spending retrieval effort on genuinely hard fields (earn rates, point values, differentiators) and flagging unsupported fields as `null` / `manual_review_needed` rather than fabricating values. Source-type round-robin (official → financial → forum → review) prevents the URL budget from collapsing into an SEO echo chamber.

Architecture & Technical Design



KEY TECH CHOICES

- **LangGraph** — state machine; node outputs immutable; errors isolated per node
- **Gemini 2.5 Flash** — query planning + structured extraction (fast, cost-effective at batch size)
- **Groq / LLaMA 3-70B** — low-latency debate advocates + judge
- **Tavily + Firecrawl** — search-then-scrape separation; Firecrawl handles PDFs natively
- **SQLite + WAL mode** — durable, portable, zero-ops persistence

DESIGN DECISIONS

- Field-level conflict detection — not object-level; catches earn-rate disagreements across different source packets
- Auto-resolve at gap > 0.20 — saves Groq calls; only truly ambiguous conflicts enter debate
- Volatility weighting: HIGH fields (earn rates) weight recency 50%; LOW fields (tier names) weight authority 50%
- SHA-256 identity hash → upsert dedup; re-scrapes update rather than duplicate
- TF-IDF cosine gate before rebuttal — skips if arguments are already too similar

ERROR HANDLING

- LLM JSON malformed → fallback to `FALLBACK_QUERIES[category]`
- Firecrawl 4xx/5xx → URL skipped, logged, pipeline continues
- Chunks <100 words filtered before SQLite write
- Judge verdict must be A / B / FLAG — cannot synthesise a new value
- Input confidence <0.90 → clarification questions before retrieval begins
- Any unsupported extraction field → `null`, never hallucinated

Results & Key Outputs

8

Schema Object Groups

30+

Canonical Fields Extracted

6

LangGraph Pipeline Nodes

≤15

Targeted Queries / Run

0.20

Auto-resolve Gap Threshold

5-step

Adversarial Debate Protocol

PROGRAM BASICS

name · brand · industry · type · geography · membership count

EARN MECHANICS

base earn rate · bonus categories · non-transactional earn

BURN MECHANICS

redemption options · thresholds · point value · expiry policy

TIER SYSTEM

tier names · qualification criteria · benefits · qualification period

PARTNERSHIPS

partners · partnership type · partnership details

DIGITAL EXPERIENCE

mobile app · app ratings · personalization · gamification

MEMBER SENTIMENT

ratings · common praise · common complaints · sources checked

COMPETITIVE POSITION

key differentiators · weaknesses · closest competitors

 PIPELINE PERFORMANCE

- Full end-to-end run: Input → Adjudicated Field Report in a single pipeline execution
- Conflict auto-resolution fires when confidence gap > 0.20 (majority of conflicts)
- 5-step debate reserved for genuinely ambiguous conflicts (gap ≤ 0.20); judge returns A/B/FLAG — never an invented value
- SHA-256 identity hash ensures re-scrapes update, not duplicate

 VALIDATED PROGRAMS (DEMO-READY)

- Marriott Bonvoy** — hotel · global · full extraction
- Delta SkyMiles** — airline · United States
- United MileagePlus** — airline · global
- Hilton Honors** — hotel · global
- Myntra Insider** — e-commerce · India
- Indigo 6E Rewards** — airline · India

KOBIE LAUNCHPAD AI HACKATHON

3 / 6

Business Impact & Activation

 WHO THIS IS BUILT FOR

- **Loyalty program strategists** — benchmark earn/burn structures across 10+ competitors in minutes, not days
- **Kobie consultants** — enter any client meeting pre-loaded with a grounded, sourced competitive brief for any program worldwide
- **Product & partnership teams** — identify partnership gaps and differentiator weaknesses at the field level
- **Analysts** — trace every claim to a source URL and access date; no black-box outputs

 THE BUSINESS CASE

- Manual competitive research: **4–8 hours per program** → ACI Agent: **<5 minutes**
- Full provenance on every claim = compliance-safe use in client deliverables
- Adjudication audit trail (advocate arguments + judge verdict) makes reasoning reviewable — not a black box
- Schema-driven design: add a new field to the config → extraction covers it on the next run with zero pipeline changes
- Temporal ordering via **retrieved_at** means stale data is flagged, not overwritten

ACTIVATION EXAMPLES — WHAT KOBIE CAN DO WITH THIS

USE CASE	FIELD(S) USED	ACTIVATION ACTION
Benchmark earn rate vs. airline peers	earn_mechanics.base_earn_rate	Pull adjudicated values for 5 competitors; generate ranked comparison table with source citations
Identify partnership whitespace	partnerships.partners	Cross-reference partner lists; flag category gaps (e.g. client has no fintech partner while 3 competitors do)
Sentiment-driven product recommendations	member_sentiment.common_complaints	Surface top complaint themes from Reddit/Trustpilot; recommend UX fixes before client redesign
Detect program rebrands / rule changes	competitive_position.key_differentiators	Re-run pipeline monthly; FLAG outcomes highlight changed values for analyst review

Live Demo Highlights

1. END-TO-END PIPELINE EXECUTION

Type any loyalty program name (e.g. *"Marriott Bonvoy"*) into the Input Verifier tab → the 6-node LangGraph pipeline fires automatically. The Pipeline Inspector tab shows live node-by-node status, query list, scraped URL count, chunk count, and per-field extraction coverage — all in one view.

2. CONFLICT ADJUDICATION IN ACTION

Open the Pipeline Inspector after a run. Expand a HIGH-volatility field (e.g. `earn_mechanics.base_earn_rate`). Show two contradicting claims from independent sources. Demonstrate auto-resolve (gap > 0.20) vs. a live 5-step adversarial debate — Advocate A → Advocate B → TF-IDF gate → Rebuttal A → Rebuttal B → LLaMA Judge returning a structured JSON verdict with `winner · deciding_factor · confidence_adjustment`.

3. COMPARE MODE — TWO PROGRAMS SIDE BY SIDE

Switch to the Compare tab. Enter two programs (e.g. *"Delta SkyMiles"* and *"United MileagePlus"*). Both pipelines run in parallel via ThreadPoolExecutor and render a structured side-by-side field comparison. Point out where adjudicated values differ and which source won each conflict.

4. CONVERSE TAB — FIELD-LEVEL Q&A

Switch to the Converse tab. Ask: *"What is the point expiry policy?"* or *"Which partners are listed?"*. Answers are grounded in stored claim evidence — not LLM memory — with source URL attribution on every response.

Learnings, Limitations & Next Steps

🎓 WHAT WE LEARNED

- **Source contradiction is the norm** — earn rates and point values conflict across 3+ sources on almost every program we tested
- The biggest quality lever was **volatility-weighted confidence scoring**, not the extraction LLM itself — weighting recency for earn rates eliminated most false auto-resolves
- Heading-based chunking dramatically outperformed fixed-window chunking for loyalty T&C pages
- Query planning with explicit field-difficulty tiers reduced Firecrawl spend ~30% vs. generic queries while improving coverage on hard fields (point values, sentiment, differentiators)

⚠️ HONEST LIMITATIONS

- Sentiment fields are sparse for smaller or regional programs — forum coverage is thin
- No **Temporal Ledger**: `retrieved_at` preserves recency ordering but there is no structured change-history audit trail
- Brief generation is scaffolded — narrator not yet wired to the adjudicated field report
- Confidence calibration tuned on a small program set; high-conflict programs with thin evidence may still misfire
- SEO echo chambers can survive dedup if the same claim propagates across many sites — lineage-based dedup not yet implemented

🚀 IF WE HAD MORE TIME...

- **Wire brief generation** to the adjudicated field report — structured analyst brief with sentence-level entailment check
- **Full Converse module** — upgrade from keyword-match QA to Groq-powered RAG over the canonical field report and brief
- **Temporal Ledger** — append-only SQLite table tracking field value changes with date stamps, enabling "what changed since last month?" queries
- **What-if simulation** — "if Program X lowered their earn rate by 0.5x, how would it compare?" modelled over adjudicated values

Shreyas R Gowda → LangGraph pipeline orchestration, extraction schema design, adjudication + 5-step debate engine | **Shreeram G Patgar** → Tavily retrieval, URL deduplication, Firecrawl integration, priority-sorted URL budgeting | **Naveen G** → Brief generation output layer, Converse module, SQLite persistence, evaluation & sentiment sourcing